СРО №2

Тема: Телефонная книга на основе связного списка (Linked List)
ГРУППА ИС 24—21

Зухуров Бехруз

🎯 Цель работы

Изучить структуру данных **связный список** и реализовать основные операции:

- добавление элементов
- поиск элементов
- удаление элементов
- вывод списка

📚 Теоретическая часть

Связный список — это динамическая структура данных, состоящая из узлов (Node), где каждый узел содержит:

- данные (имя и номер телефона)
- ссылку на следующий элемент

Каждый элемент хранится отдельно в памяти и связан с другим через ссылки.

📁 Структура программы

Программа состоит из трёх частей:

- **Main** — управление меню и взаимодействие с пользователем
- **Node** — структура одного контакта
- **PhoneBook** — логика работы со связным списком

⚙ Основные операции

➕ Добавление

Новый элемент добавляется в конец списка.
Если список пустой — он становится первым элементом.

🔍 Поиск

Происходит последовательный перебор всех элементов, пока не найден нужный контакт.

❌ Удаление

Удаление выполняется путём изменения ссылок:

- если удаляется первый элемент — меняется head
- если внутри списка — ссылка предыдущего узла перепрыгивает удаляемый

📋 Вывод

Все элементы списка выводятся по очереди с начала до конца.

💻 Практическая часть

```
import java.util.Scanner;

public class cpo2_Main {

    public static void main(String[] args) {

        PhoneBook book = new PhoneBook();
        Scanner sc = new Scanner(System.in);

        while (true) {

            System.out.println("\n===== PHONE BOOK =====");
            System.out.println("1 - Add contact");
            System.out.println("2 - Search contact");
            System.out.println("3 - Delete contact");
            System.out.println("4 - Show all");
            System.out.println("5 - Exit");
            System.out.print("Choice: ");
```

```
            int choice = sc.nextInt();
            sc.nextLine();

            if (choice == 1) {

                System.out.print("Enter name: ");
                String name = sc.nextLine();

                System.out.print("Enter number: ");
                String number = sc.nextLine();

                book.add(name, number);

                System.out.println("✓ Contact added");

            } else if (choice == 2) {

                System.out.print("Enter name: ");
                String name = sc.nextLine();

                Node result = book.search(name);

                if (result != null) {
                    System.out.println("✓ Found: " + result.name + " -> " +
result.number);
                } else {
                    System.out.println("✗ Not found");
                }

            } else if (choice == 3) {

                System.out.print("Enter name to delete: ");
                String name = sc.nextLine();

                if (book.delete(name)) {
                    System.out.println("✓ Deleted");
                } else {
                    System.out.println("✗ Not found");
                }

            } else if (choice == 4) {

                book.showAll();

            } else if (choice == 5) {

                System.out.println("Bye!");
                break;
            }
        }

        sc.close();
    }
}

/* ===== NODE (элемент списка) ===== */
class Node {
```

```java
    String name;
    String number;
    Node next;

    Node(String name, String number) {
        this.name = name;
        this.number = number;
        this.next = null;
    }
}

/* ===== PHONE BOOK (связный список) ===== */
class PhoneBook {

    private Node head;

    // добавление
    public void add(String name, String number) {

        Node newNode = new Node(name, number);

        if (head == null) {
            head = newNode;
            return;
        }

        Node temp = head;

        while (temp.next != null) {
            temp = temp.next;
        }

        temp.next = newNode;
    }

    // поиск
    public Node search(String name) {

        Node temp = head;

        while (temp != null) {
            if (temp.name.equalsIgnoreCase(name)) {
                return temp;
            }
            temp = temp.next;
        }

        return null;
    }

    // удаление
    public boolean delete(String name) {

        if (head == null) return false;

        if (head.name.equalsIgnoreCase(name)) {
            head = head.next;
            return true;
        }
```

```java
        Node temp = head;

        while (temp.next != null) {

            if (temp.next.name.equalsIgnoreCase(name)) {
                temp.next = temp.next.next;
                return true;
            }

            temp = temp.next;
        }

        return false;
    }

    // вывод всех
    public void showAll() {

        if (head == null) {
            System.out.println("Phone book is empty");
            return;
        }

        Node temp = head;

        while (temp != null) {
            System.out.println(temp.name + " -> " + temp.number);
            temp = temp.next;
        }
    }
}
```

)